

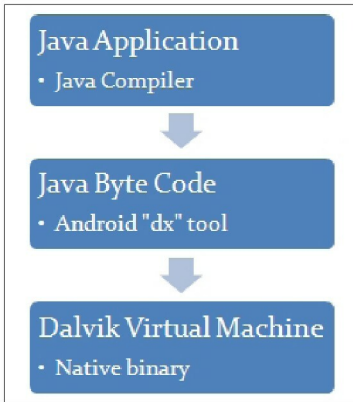


Figure 3: The compilation flow of a Java application on Android

libraries and the Android run-time. The common set of system libraries includes:

- System C library (libc)
- Media library
- SQLite (database engine)
- SGL (to support 2-D graphics)

The Android runtime is a combination of the Dalvik VM and Java core libraries.

The Linux kernel is the last layer in the software stack, and provides an abstraction of the underlying hardware, also taking care of networking, process and memory management.

Dalvik VM

Dalvik VM is Google's implementation of the Java VM, targeted to run on mobile devices. Dalvik is a process VM; each Android application runs in its own Linux process. Every process on Android has its own instance of the Dalvik VM, which is created along with the process, and destroyed during process termination.

'init' is the parent process for all Android processes. During start-up (see Figure 2), in addition to a handful of daemons, 'init' forks a process named 'zygote', which loads and initialises an instance of the Dalvik VM and core services. Zygote then creates a read socket, and runs a select loop on it. When we run an application, a message is sent to Zygote, which forks, and contacts the Android Activity manager service to run the application's Java class. Thus, all processes share a copy of DVM, created by the Zygote process, and avoid keeping duplicate pages of DVM.

Actually, pages for DVM are COW (copy-on-write) among processes; as soon as a process writes to a shared, read-only page, that page becomes local to the process.

Why a process VM?

This design has many benefits compared to a single system VM instance that serves all processes in the system. If the VM crashes, the whole system would crash, killing all processes. In Android, this problem has been narrowed down to the granularity of a single process.

VM internal details

The Dalvik VM accepts Dalvik byte-code (*.dex files) as input, instead of the Java byte-code expected by a JVM. The *.dex files are created by the 'dx' tool, which is part of the Android system (see Figure 3). This tool takes Java class files created by the Java compiler as the input.

How is a Dalvik-like register VM implemented?

To answer that, let's implement a register VM in C. The primary components of a register VM are:

1. A register set: Simulated with an array of integers acting as read/write memory cells.
2. An instruction set: What operations, and how many operands, should we support?
 - Our primitive VM supports addition, load and halt operations.
 - Addressing mode: Operands are part of the instructions, so an instruction decode would unveil the associated operands.
3. Execution unit: The engine works similar to a CPU:
 - It fetches the next instruction from the program.
 - It decodes the instruction to figure out the type of operation and operands.
 - It executes the decoded instruction.

The code for the VM is too large to include in the article, and so is in the file `myvm.c`, which you can find at http://linuxforu.com/article_source_code/june11/myvm.c 

References

- [1] <http://www.dalvikvm.com/>
- [2] <http://www.lua.org/>
- [3] http://en.wikipedia.org/wiki/Stack_machine#Virtual_stack_machines
- [4] http://www.userix.org/events/vee05/full_papers/p153-yunhe.pdf

By: Vishal Kanaujia

The author is a member of the technical staff at NetApp, Bengaluru. He is interested in compiler development, performance tuning, and algorithms. Do send your comments and feedback to vishalkanaujia@gmail.com.